The initial pipeline suffered from data leakage and overfitting, leading to inflated performance. After implementing cross-validated feature selection, stricter regularization, and model-specific feature tuning, obtained stable and generalizable results. The final model achieves AUC ~0.87, consistent with published literature.

Initial Pipeline:

- Extracted k-mer features from raw genome sequences → 4096 features per genome
- Split data 80/20 train/test with stratification
- Selected top 300 features using ANOVA (k=2000 pre-filter) + Random Forest importance
- Tuned LR, RF, XGBoost with GridSearchCV (scoring: ROC-AUC)
- Trained and evaluated all three models

Model	Train AUC	Test AUC	Gap
LR	0.973	0.917	0.056
RF	0.998	0.889	0.109 (high)
XGB	0.999	0.913	0.086 (high)

What went wrong and why:

RF and XGB were essentially memorizing the training data. Train AUC of 0.998–0.999 on any real dataset means the model has overfit, not learned. The root causes were:

- Data leakage in feature selection — the RF used for importance ranking was trained on the full training set, then selected features from that same data. Any model evaluated later on those features was seeing a biased feature set
- 300 features was too many for 2628 samples in tree models — the sample-to-feature ratio was only 8.7x, far too low
- max_depth=15 for RF allowed trees deep enough to memorize individual genome signatures

- reg_lambda=1 for XGB was insufficient regularization for high-dimensional k-mer data

First Fix (Plugging the Leakage)

What changed:

- Replaced single-RF importance with **5-fold cross-validated permutation importance** — the RF in each fold was evaluated on held-out validation data it had never seen, eliminating the leakage
- Replaced `SelectKBest(k=2000)` with `SelectFpr(alpha=0.01)` — features kept by statistical significance rather than arbitrary count
- Reduced top features from 300 → 150
- Added stricter RF parameters: max_depth lowered, min_samples_leaf raised
- Added `reg_alpha` (L1) to XGB which was completely missing before
- Changed evaluation to include **MCC and PR-AUC** alongside AUC and F1
- Added confusion matrix with per-class breakdown

Model	Test AUC	Gap
LR	0.838	0.049
RF	0.889	0.109 (high)
XGB	0.900	0.093 (high)

The leak fix worked — numbers became more honest. LR stabilized beautifully. But RF and XGB were still overfitting because 150 features remained too many for tree models on this sample size, and the tuning grid hadn't searched aggressive enough regularization.

third Fix (Model-Specific Feature Counts + Threshold Tuning):

The key insight was that LR and tree models have fundamentally different relationships with feature count. A linear model with L2 regularization handles many features well because regularization shrinks irrelevant coefficients toward zero. Tree models overfit because every feature is a potential split point — more features means more ways to memorize.

with 2628 training samples: LR → 80 features → 26x sample/feature ratio (fine for linear)
RF → 40 features → 53x sample/feature ratio (good for trees)
XGB → 30 features → 70x sample/feature ratio (good for boosting)

also changed:

- ANOVA alpha tightened: 0.01 → 0.001 (stricter pre-filter for 4096-feature space)
- Tuning scoring changed from `roc_auc` → `recall` to directly target the FN problem
- Added **decision threshold tuning** — instead of always predicting Resistant at probability ≥ 0.5, each model searched thresholds 0.25–0.65 to maximize recall while keeping precision above 0.70.

Model	Train AUC	Gap	Resistant Recall
LR	0.811	0.040	0.818(tuned)
RF	0.853	0.102(high)	0.741
XGB	0.876	0.093(high)	0.787

LR's resistant recall improved from 0.719 → 0.818, meaning 26 fewer missed resistant genomes. But a new problem appeared — tuning for recall alone collapsed LR's precision to 0.68, meaning too many susceptible genomes were being falsely flagged as resistant. This showed that optimizing a single metric in isolation is dangerous.

Final Pipeline (Balanced Optimization)

What changed:

- Scoring changed from `recall` → **F1** — forces the tuner to balance precision and recall simultaneously
- Threshold tuning criteria tightened: must achieve recall ≥ 0.75 **and** precision ≥ 0.70 simultaneously before a threshold is accepted
- Each model now trains, tunes, and evaluates on its own dedicated feature set
- Forced hard floors on regularization in `train_models.py` regardless of what GridSearch returned — because GridSearch on training folds still rewards mild memorization

After running applying final forced regularization (max_depth=4, min_samples_leaf=25 for RF; reg_lambda=30, min_child_weight=25 for XGB):

Model	**features**	**Test AUC**	**PR-AUC**	**MCC**	**AUC Gap**
LR	80	0.873	0.864	0.597	0.051
RF	40	0.859	0.859	0.557	0.0814
XGB	**30**	0.8753	0.886	0.555	0.0694